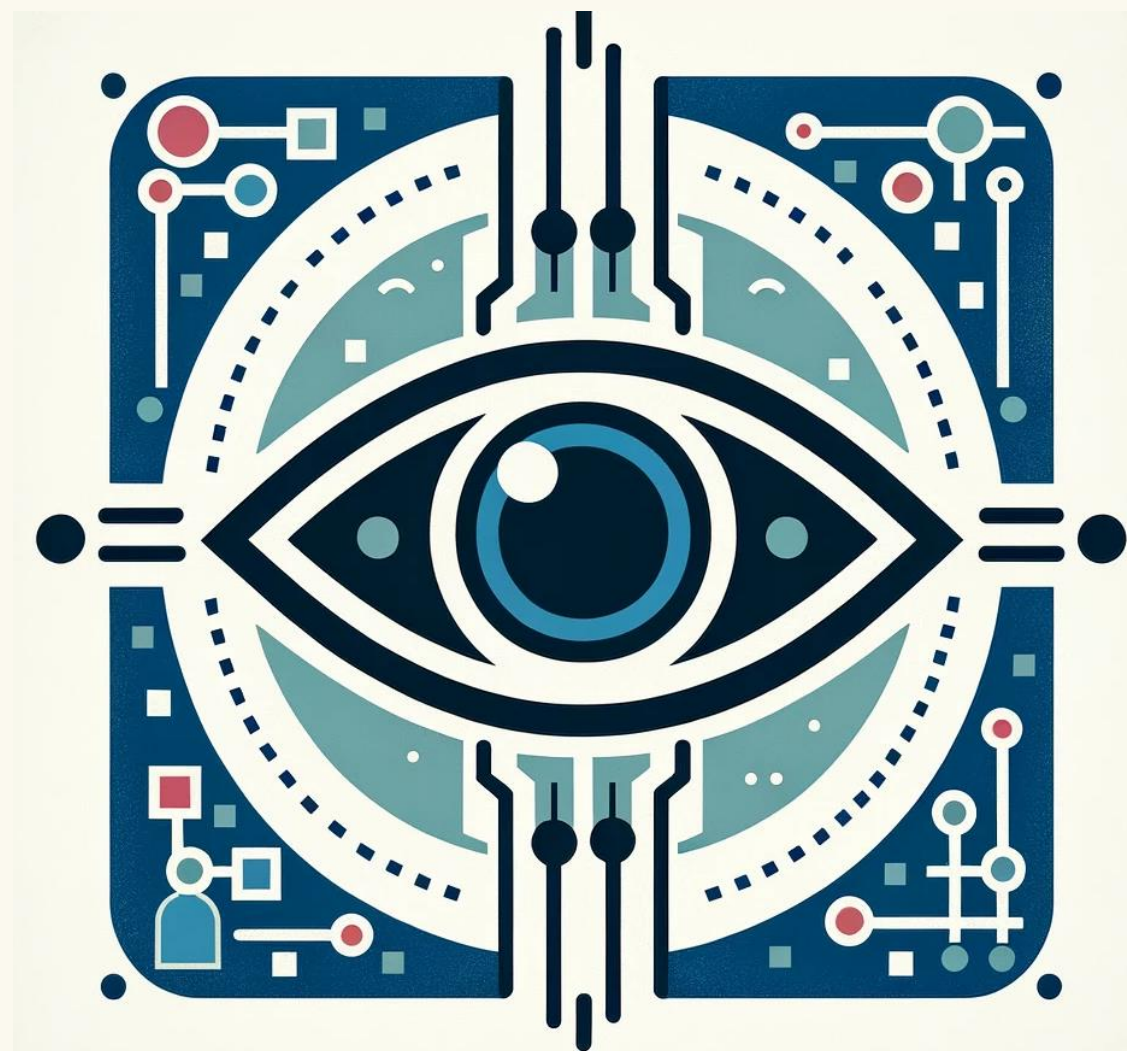
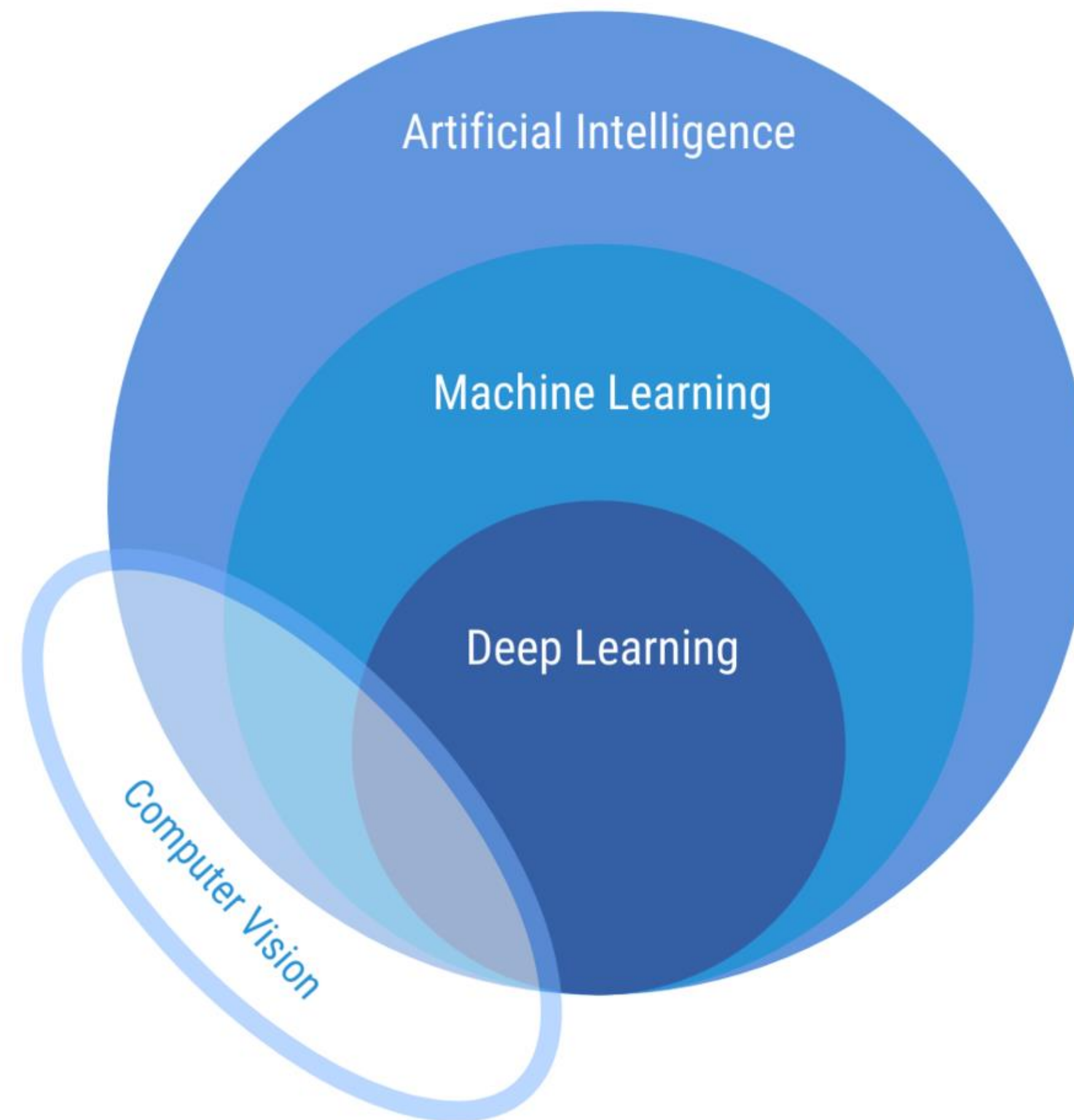


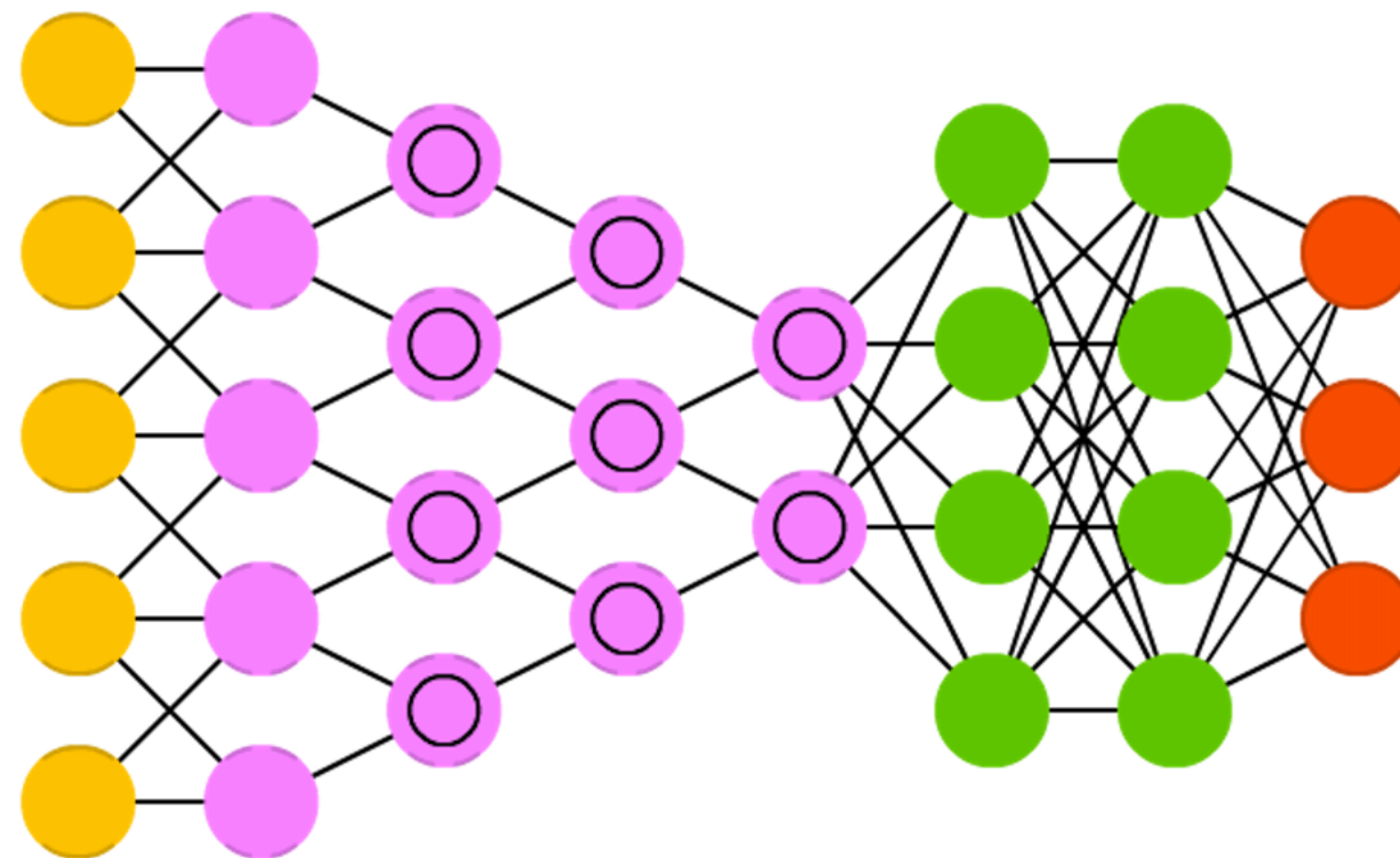


# COMPUTER VISION

## Convolutional Neural Networks

Now, let us focus on deep learning

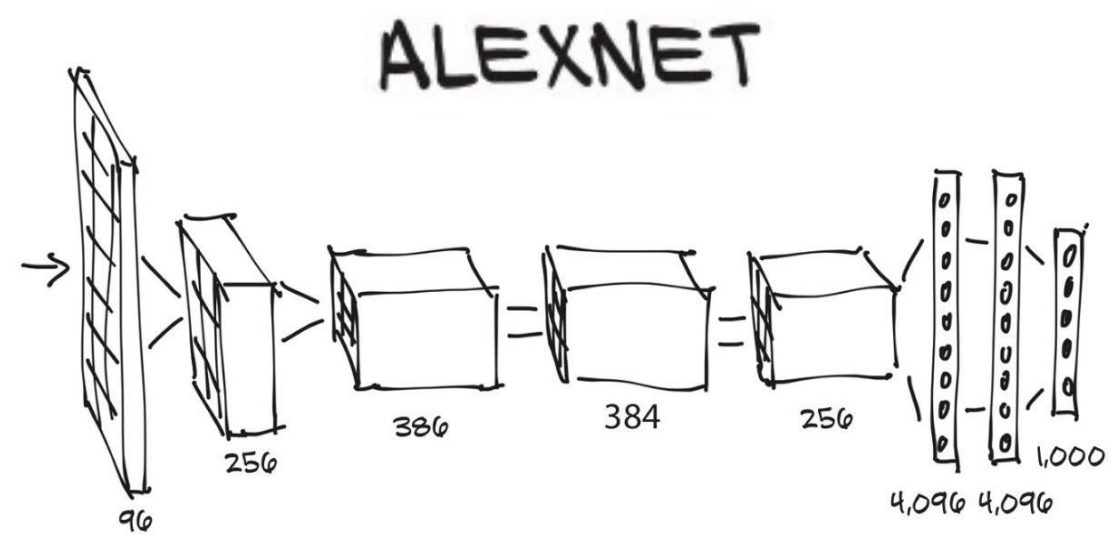




Deep learning → very large and deep neural networks

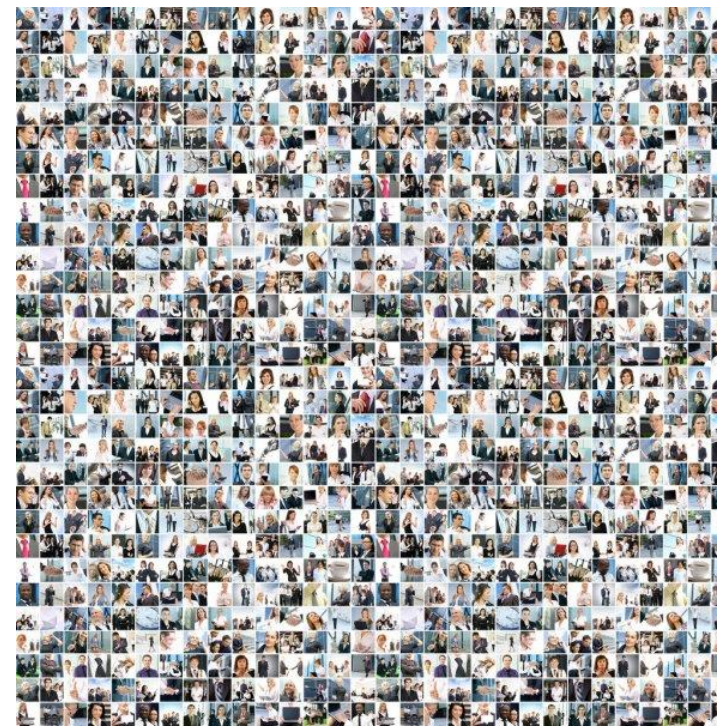


# Deep Learning Revolution



Better Architectures and  
Learning Strategies

+



A Loooooot More Data

+



Processing Power





IMGENET

[www.image-net.org](http://www.image-net.org)

**22K** categories and **14M** images

- Animals
  - Bird
  - Fish
  - Mammal
  - Invertebrate
- Plants
  - Tree
  - Flower
  - Food
  - Materials
- Structures
  - Artifact
    - Tools
    - Appliances
    - Structures
- Person
  - Scenes
    - Indoor
    - Geological Formations
  - Sport Activities



Deng, Dong, Socher, Li, Li, & Fei-Fei, 2009



# IMAGENET Large Scale Visual Recognition Challenge

Steel drum

The Image Classification Challenge:

1,000 object classes

1,431,167 images



**Output:**

Scale

T-shirt

Steel drum

Drumstick

Mud turtle



**Output:**

Scale

T-shirt

Giant panda

Drumstick

Mud turtle

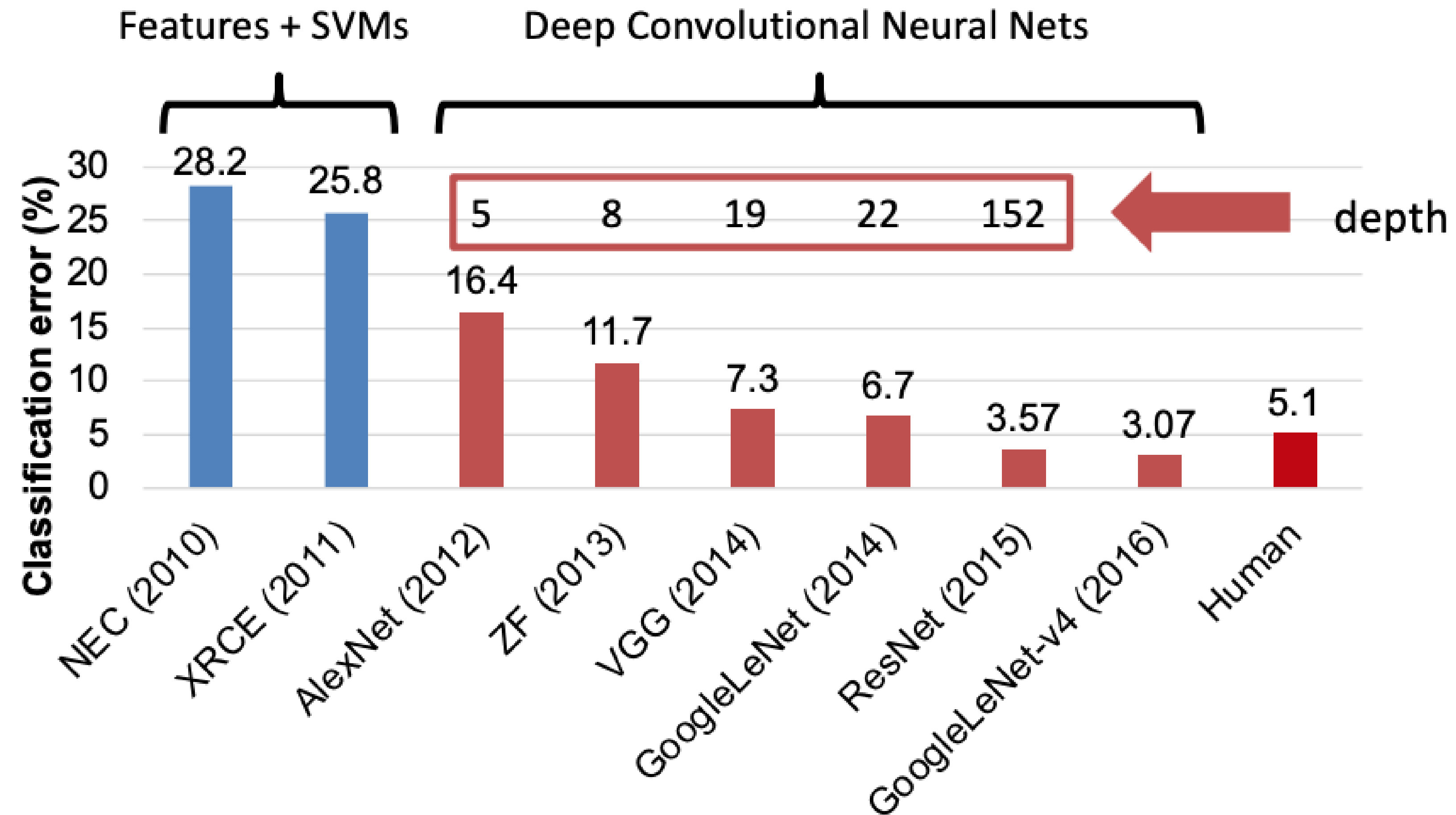


Russakovsky et al. arXiv, 2014



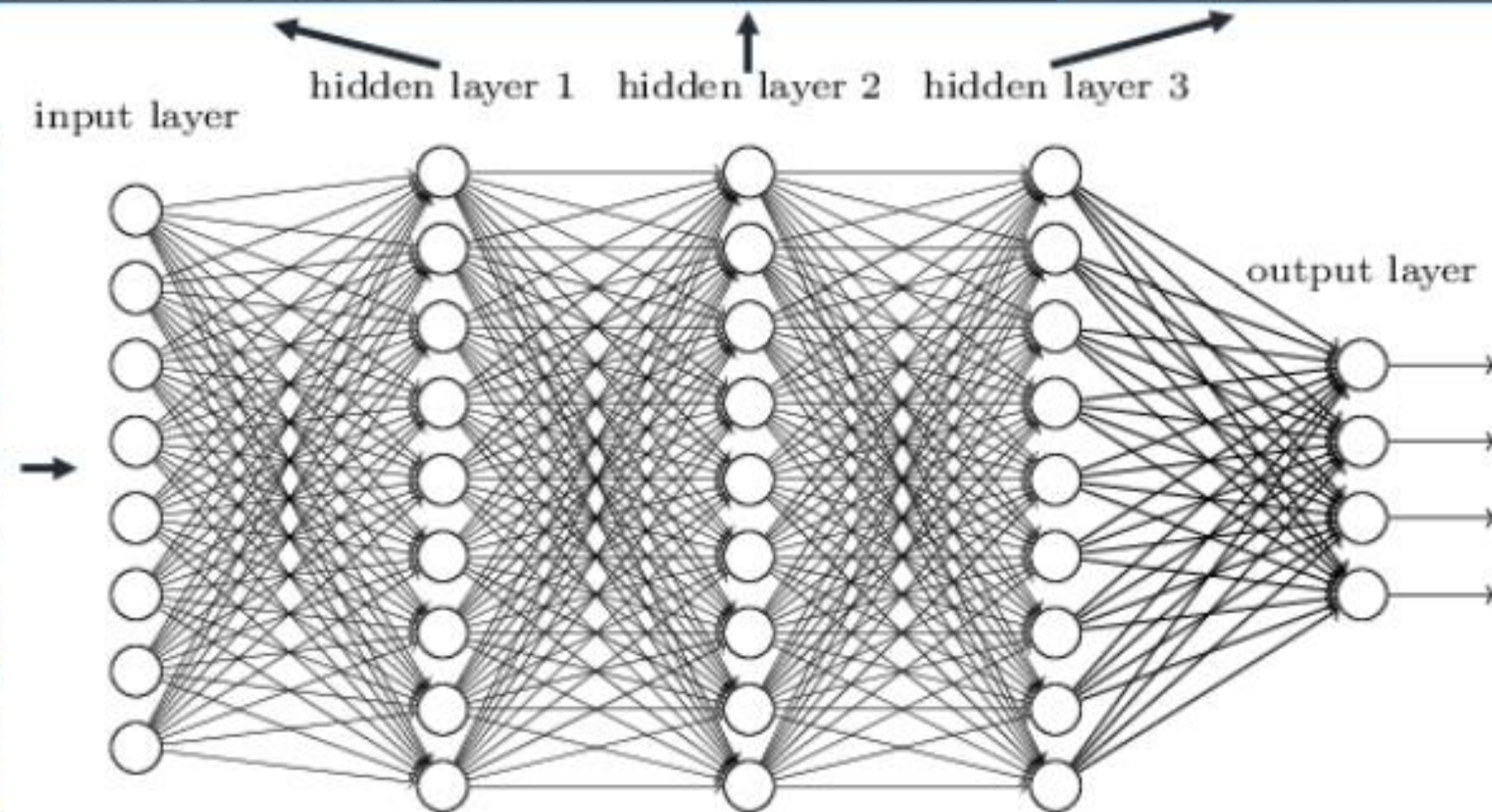
# Image classification

ImageNet Large Scale Visual Recognition Challenge

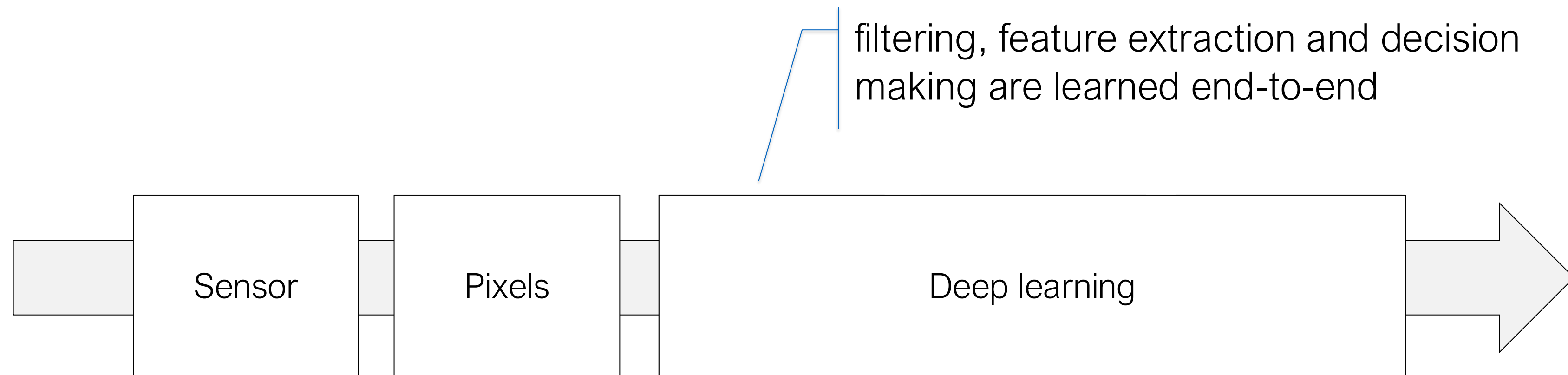


# The power of depth

Deep neural networks learn hierarchical feature representations







Deep learning pipeline



# Deep Learning Libraries

- TensorFlow: <https://www.tensorflow.org/>
- Keras: <http://keras.io>
- PyTorch: <http://pytorch.org>
- MXNet: <https://mxnet.apache.org/>

- Architectures easy to create
- CPU/GPU
- Pre-existing models





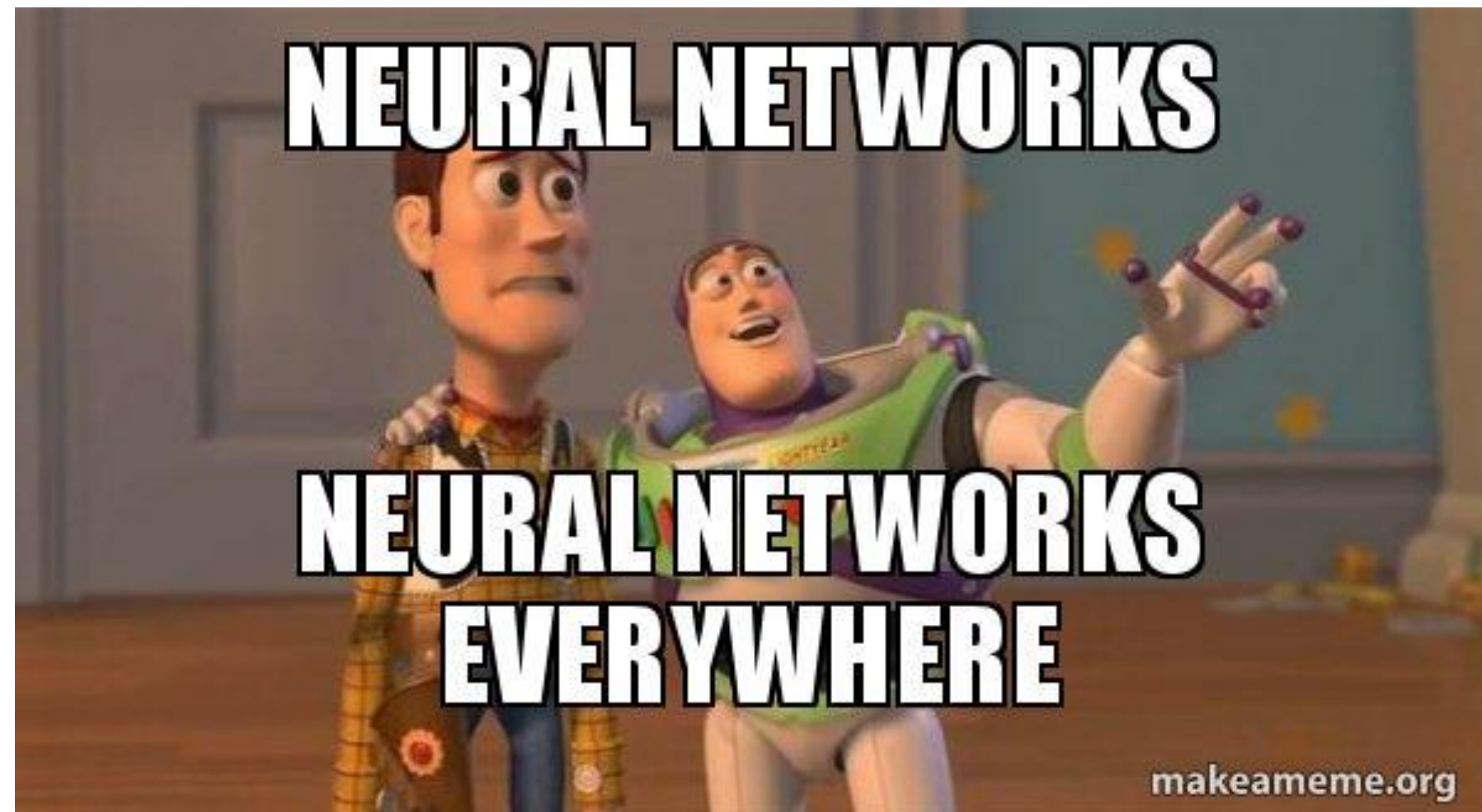
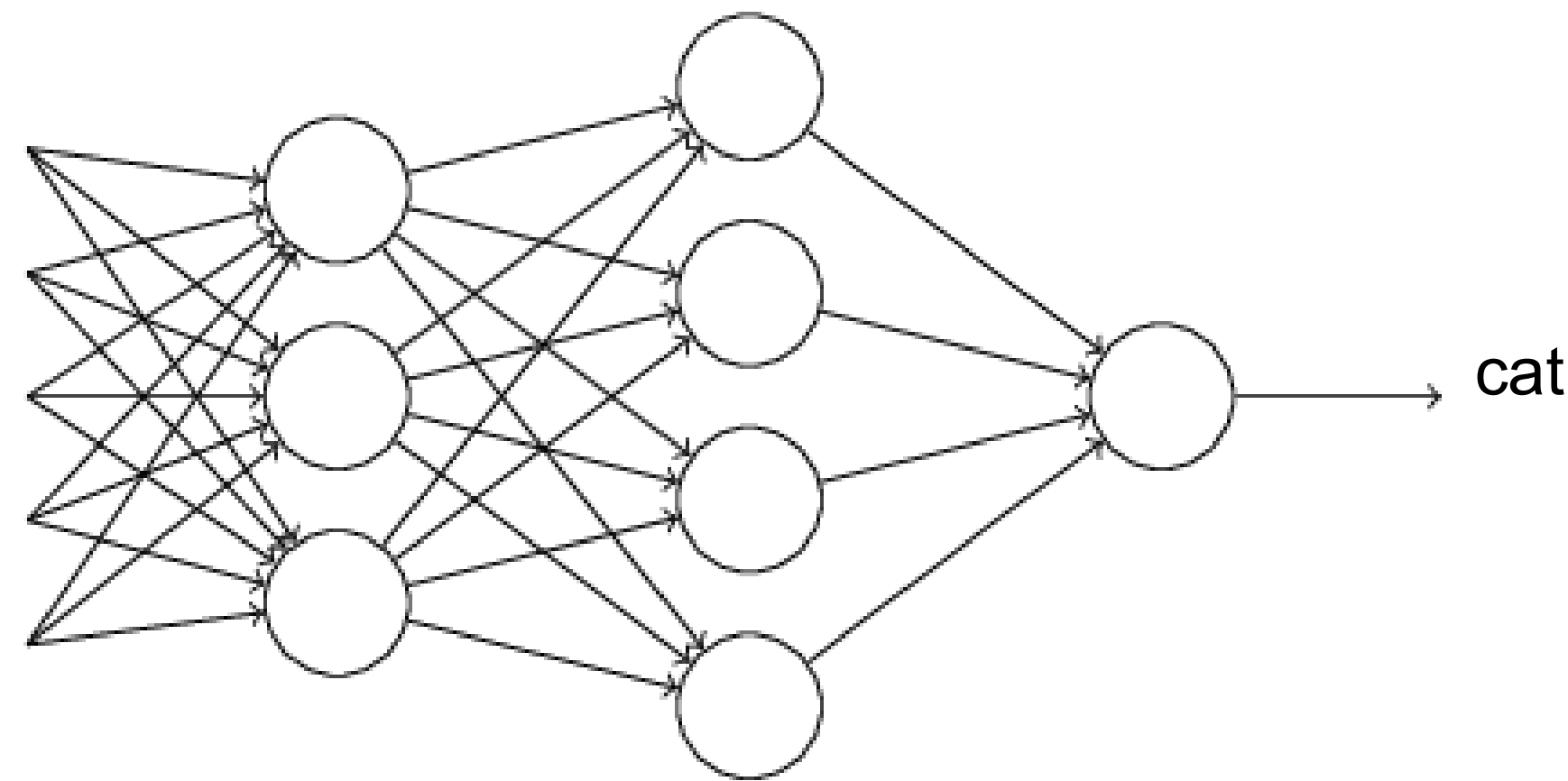


Image source: <https://dudeperf3ct.github.io/>

# Using MLPs with Images

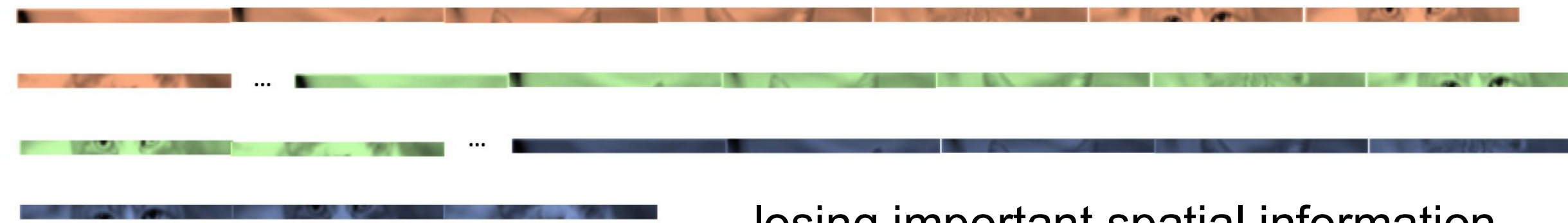


Can we make this work?



# Using MLPs with Images

- Image needs to be represented as a long vector...



... losing important spatial information.

- A weight vector that learns to match a specific feature in cat images will not match the same feature in the same but shifted or scaled image  
→ The MLP is not invariant to image translation or scaling
- Also, the MLP model has many weights that are not learning anything useful for images

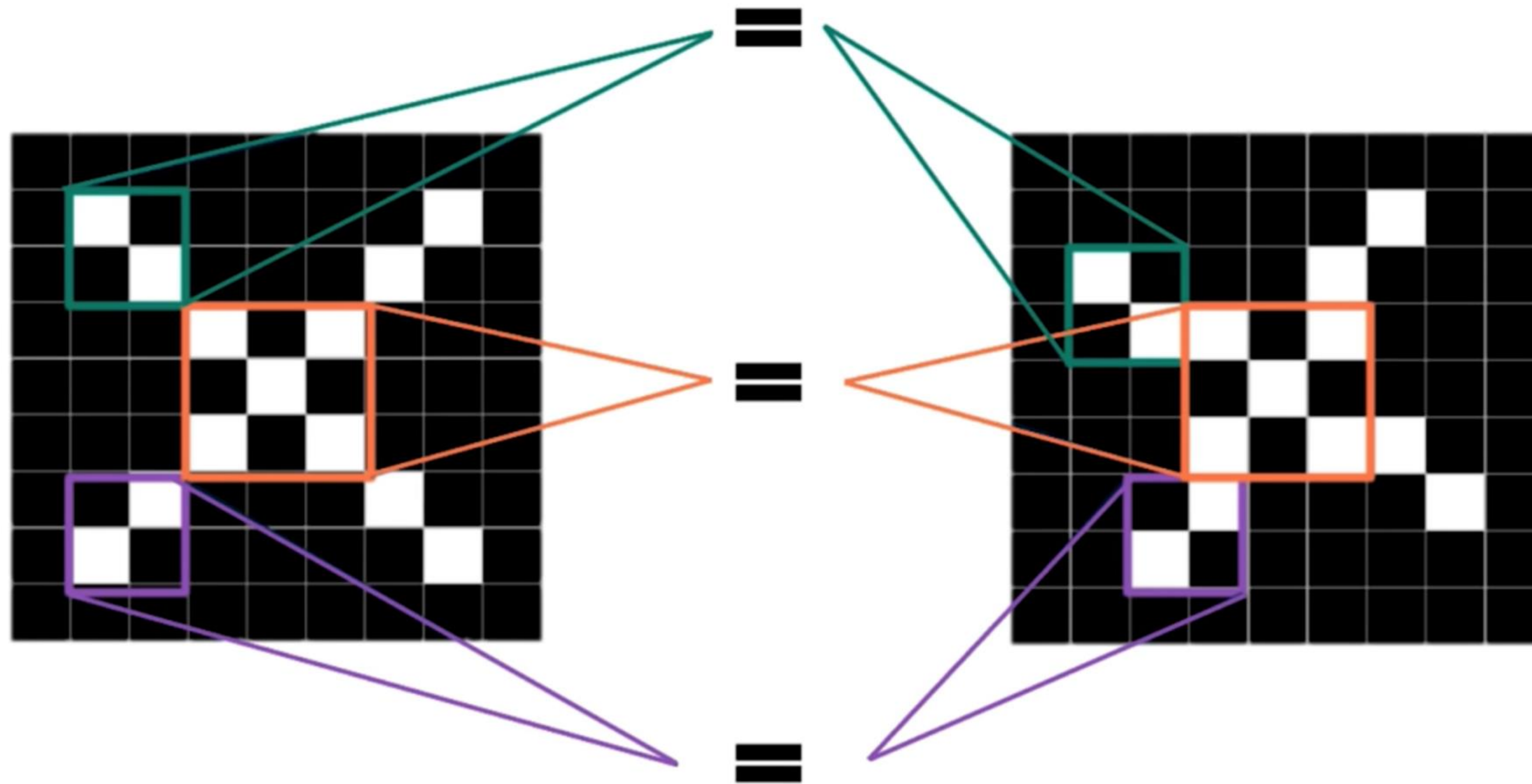
# Interpreting Images



We want to be able to classify an object in the image, even if shifted, resized, rotated, deformed.

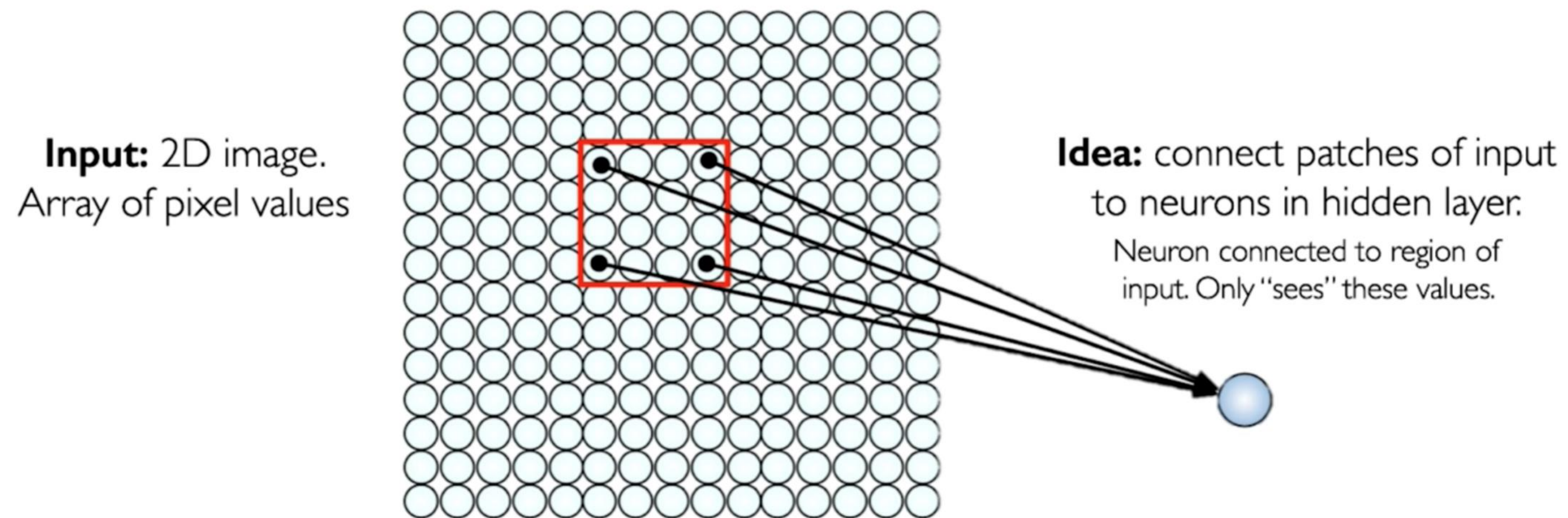


# Interpreting Images



Instead of comparing the whole image, we should compare local features

# Using Spatial Structure

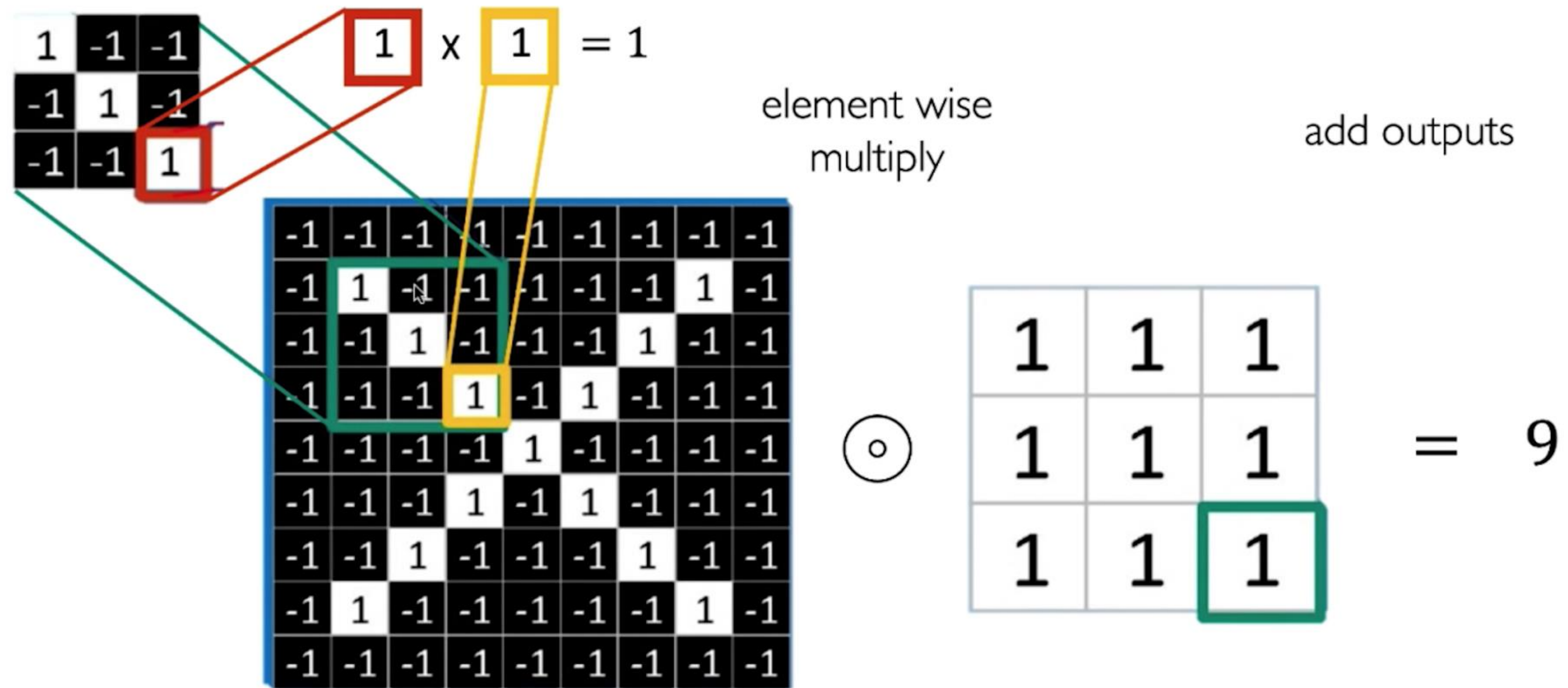


But, how do we analyse the complete image?

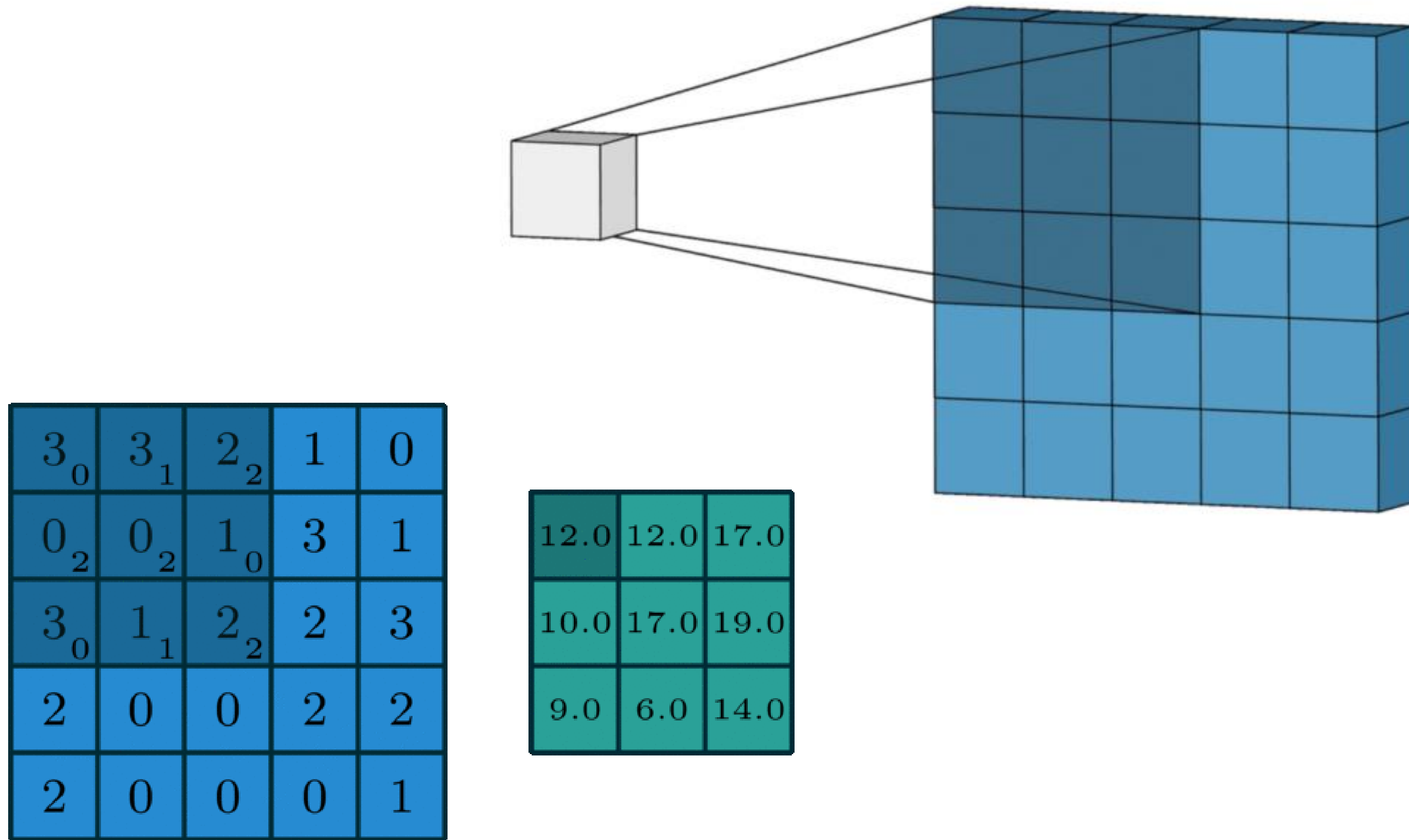
- Multiple neurons with shared weights
- In practice, we use a sliding window to go through the image



# Convolution



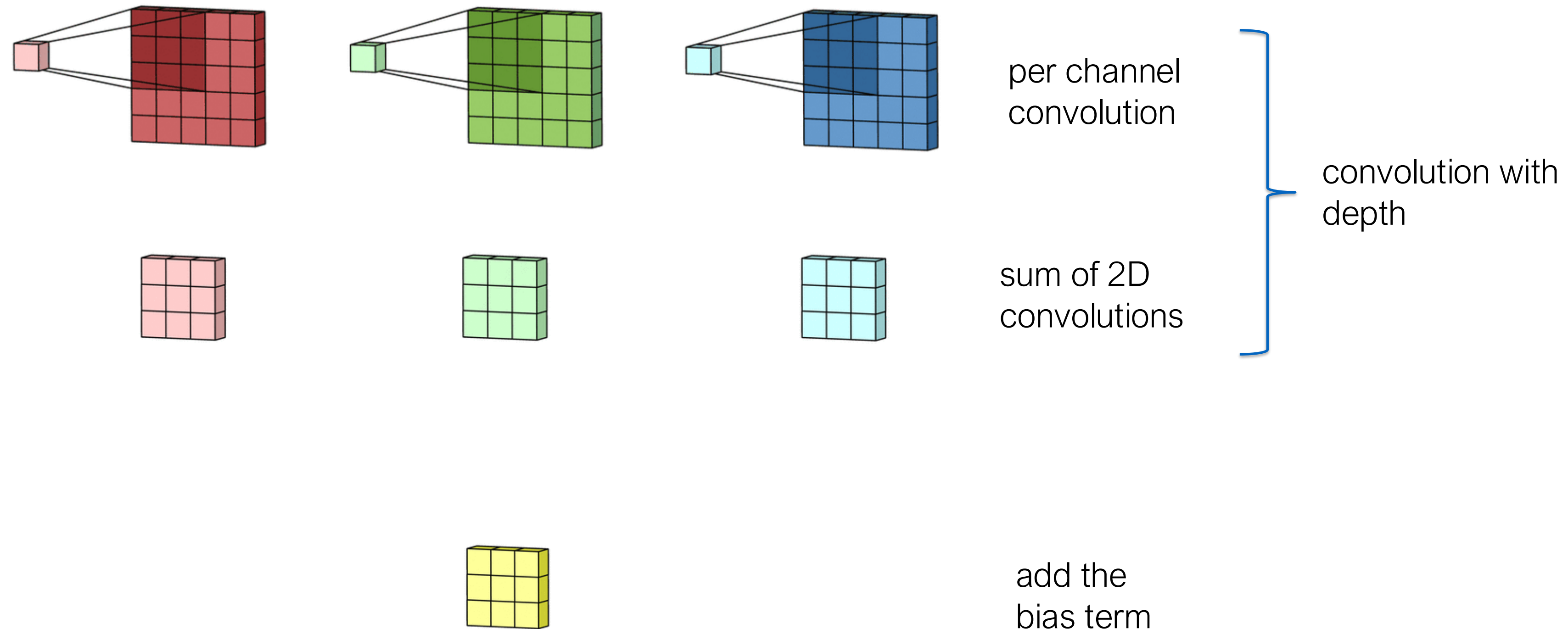
# Convolution Layer





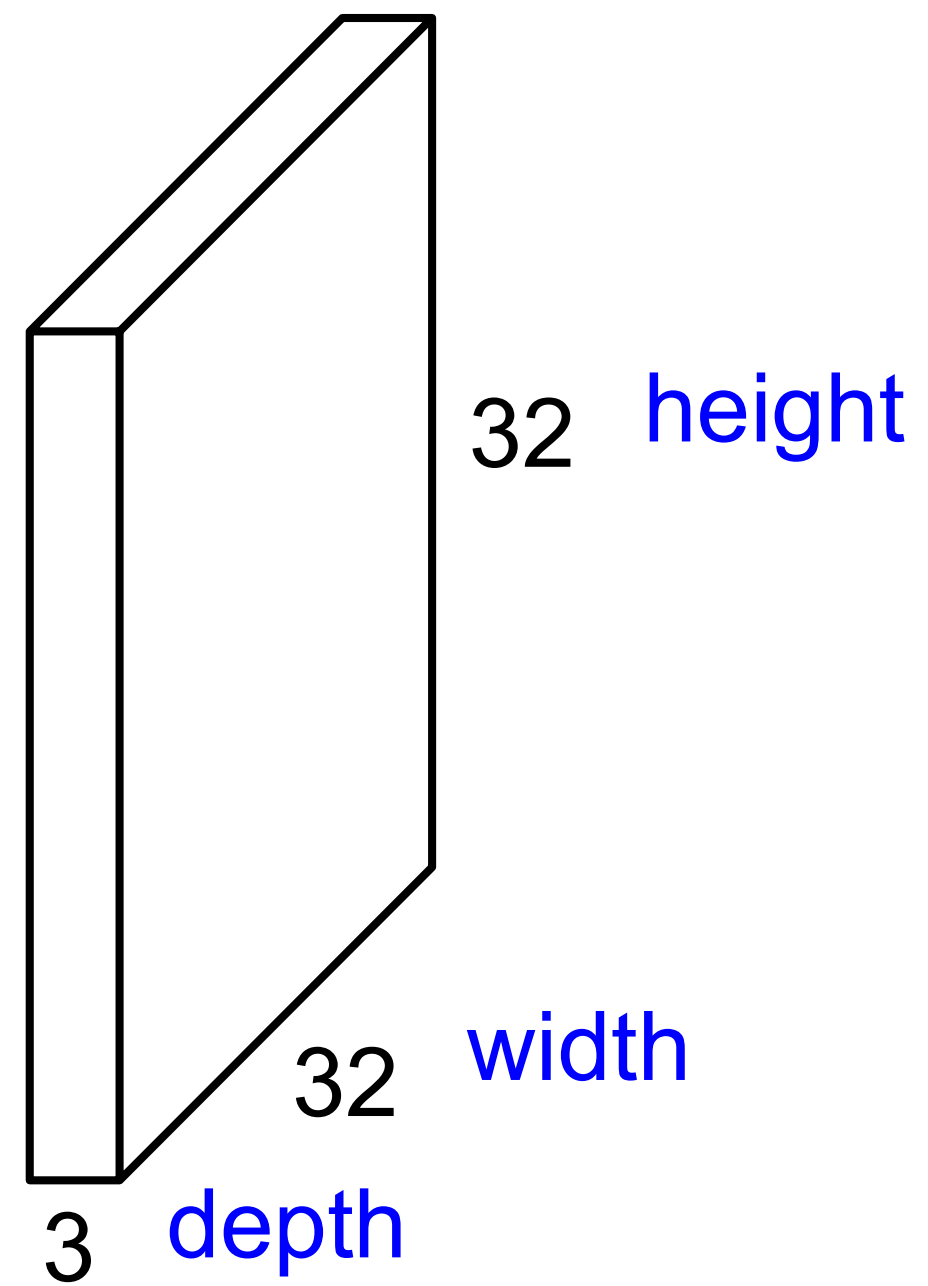
# Convolution Layer

Channels of the input image (usually RGB) are represented as depth of a 3D block  
→ Filters' outputs are also represented as channels in the depth dimension



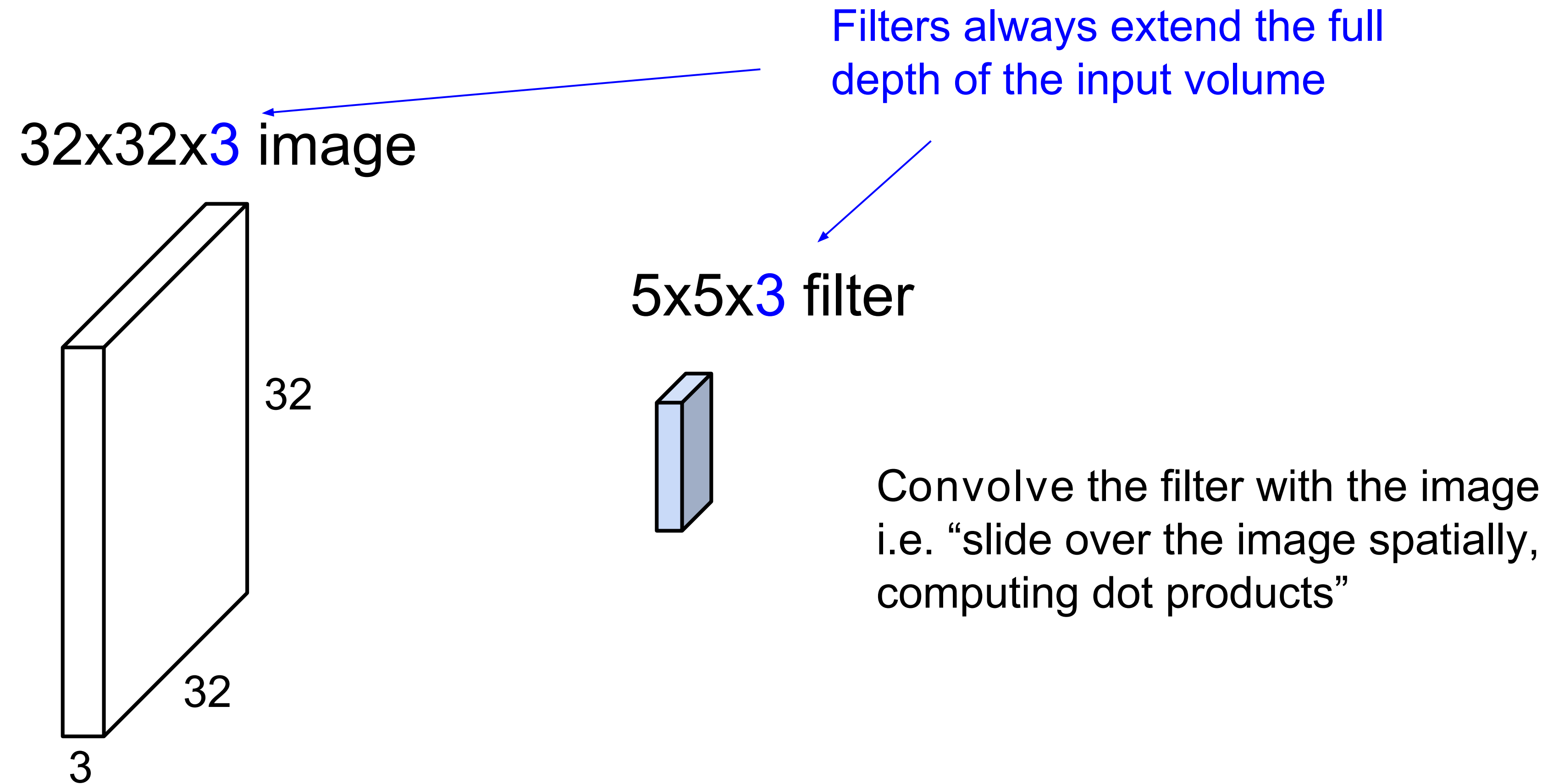
# Convolution Layer

32x32x3 image

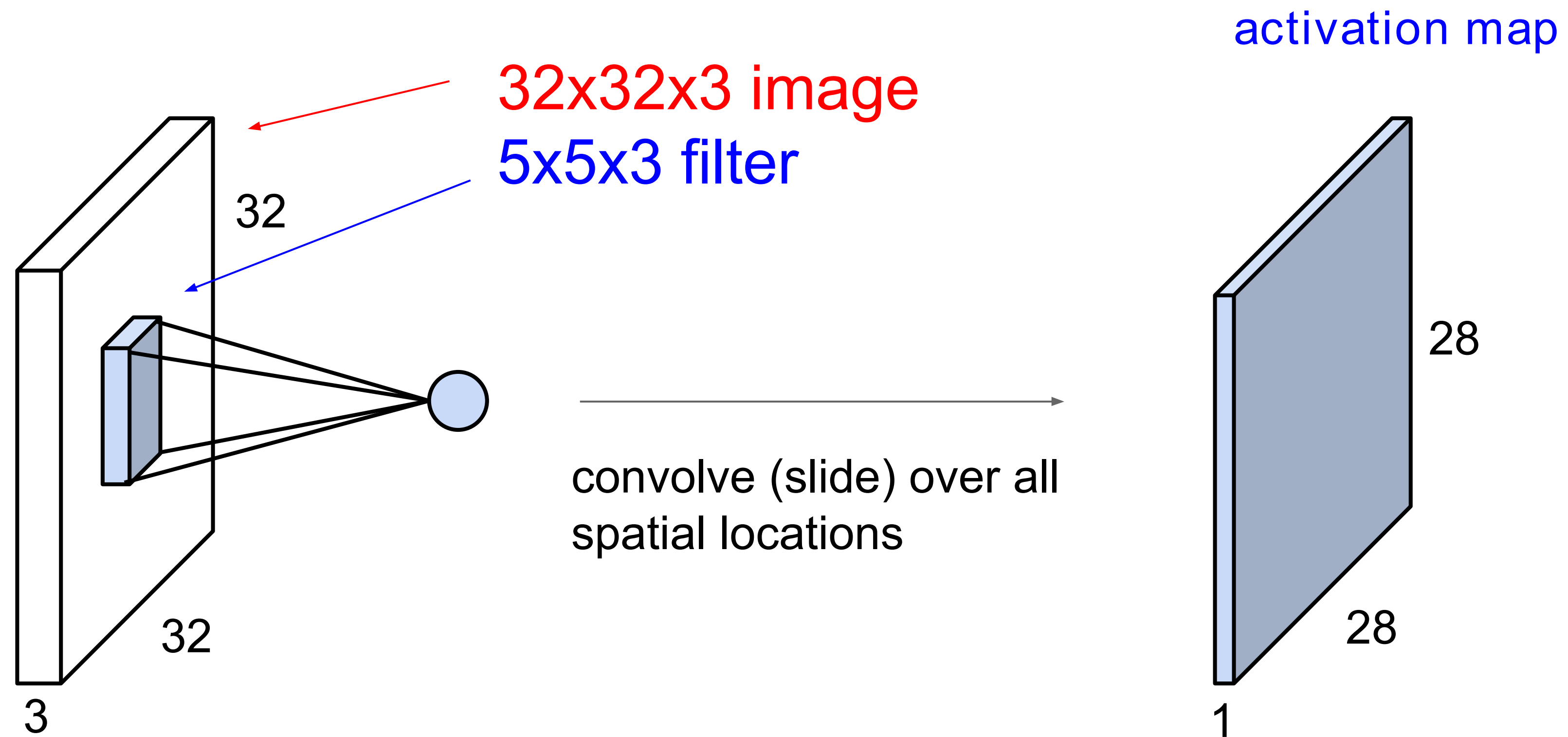




# Convolution Layer



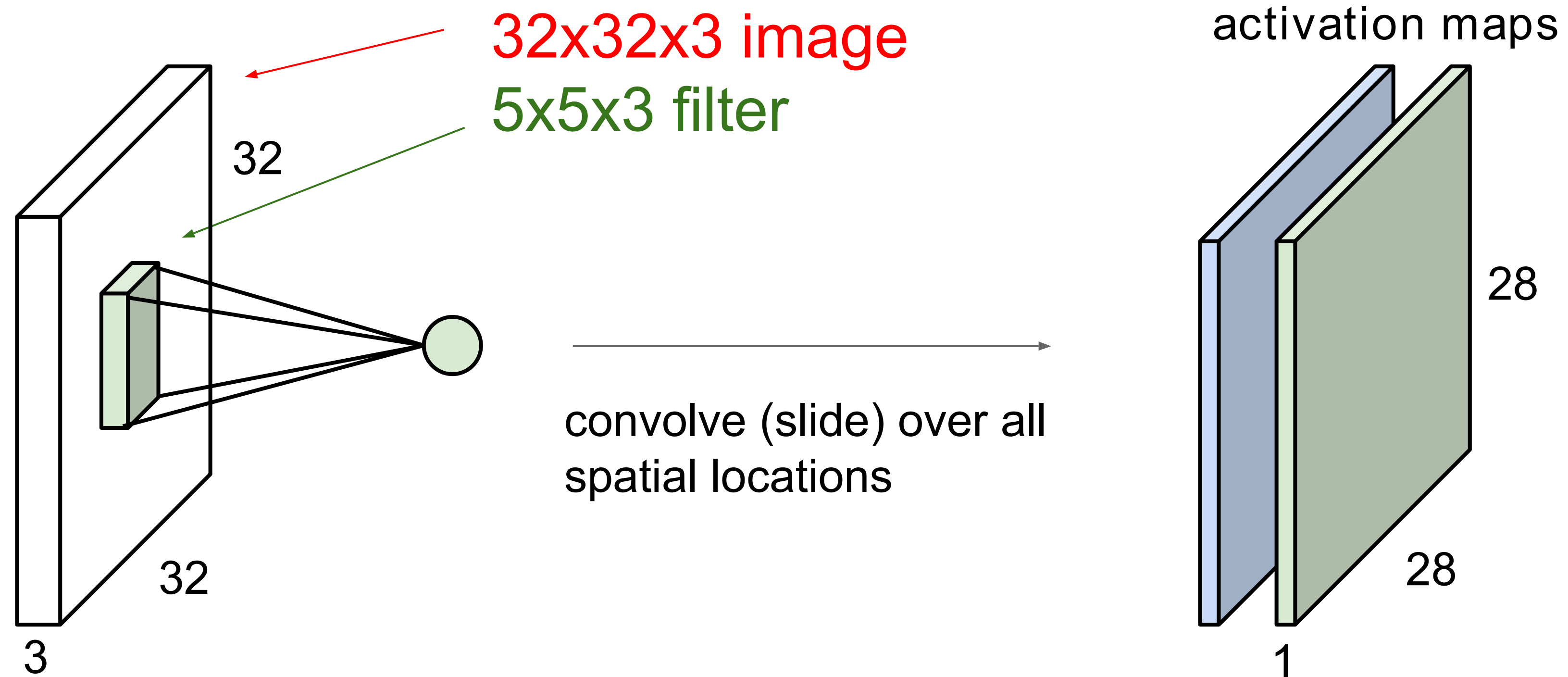
# Convolution Layer





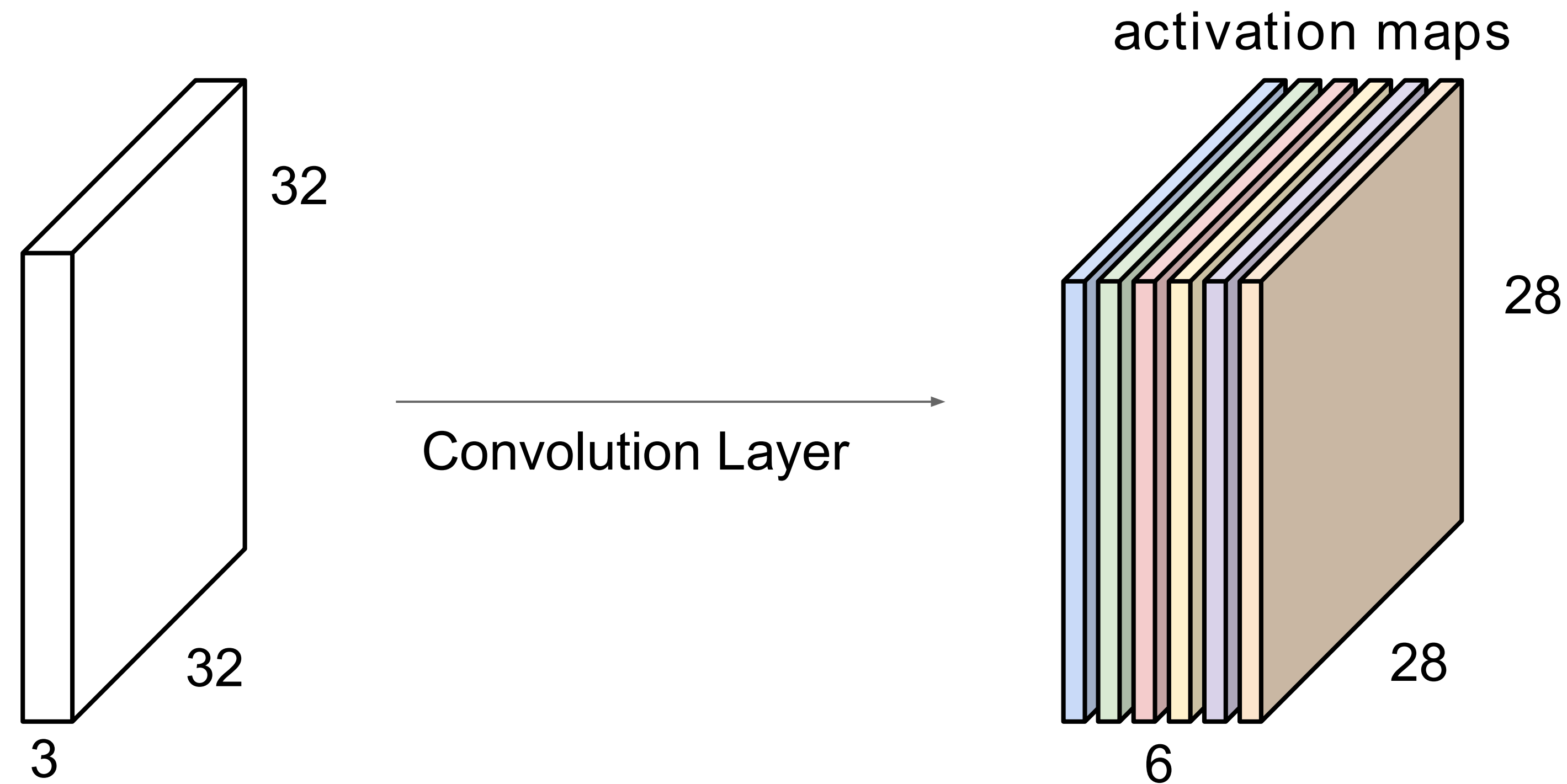
# Convolution Layer

consider a second, **green** filter



# Convolution Layer

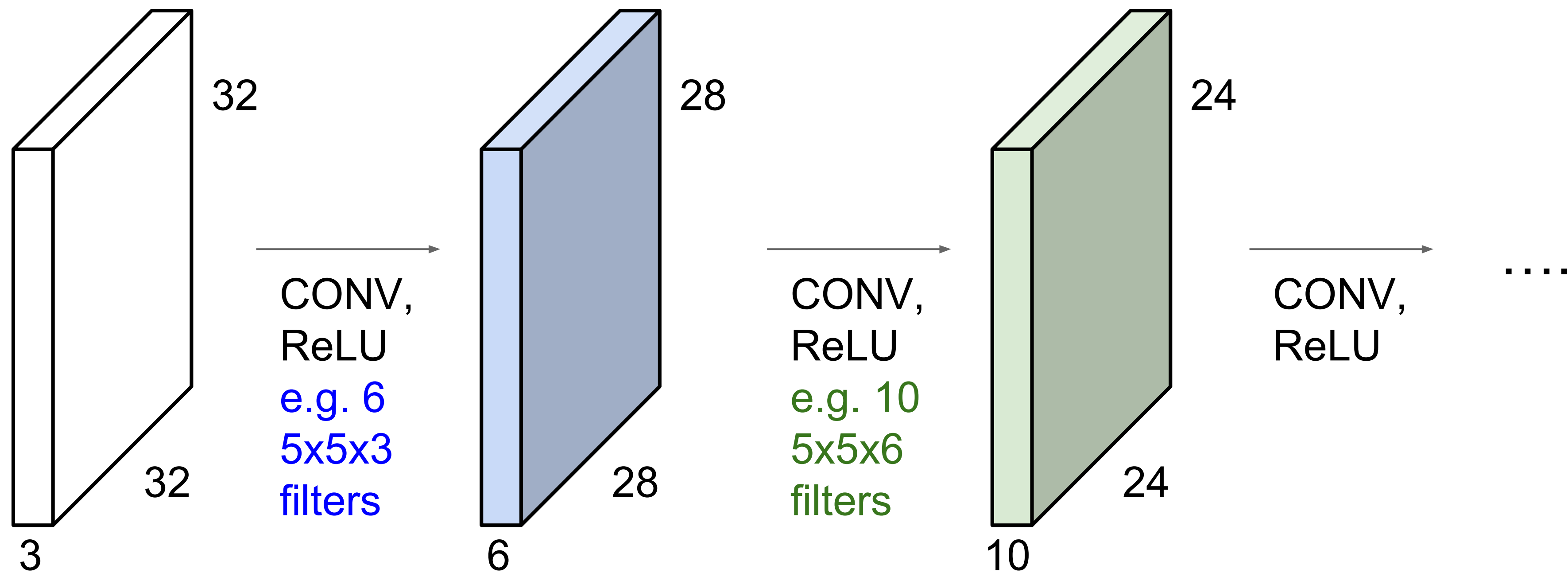
For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



We stack these up to get a “new image” of size 28x28x6!

# Convolution Layer

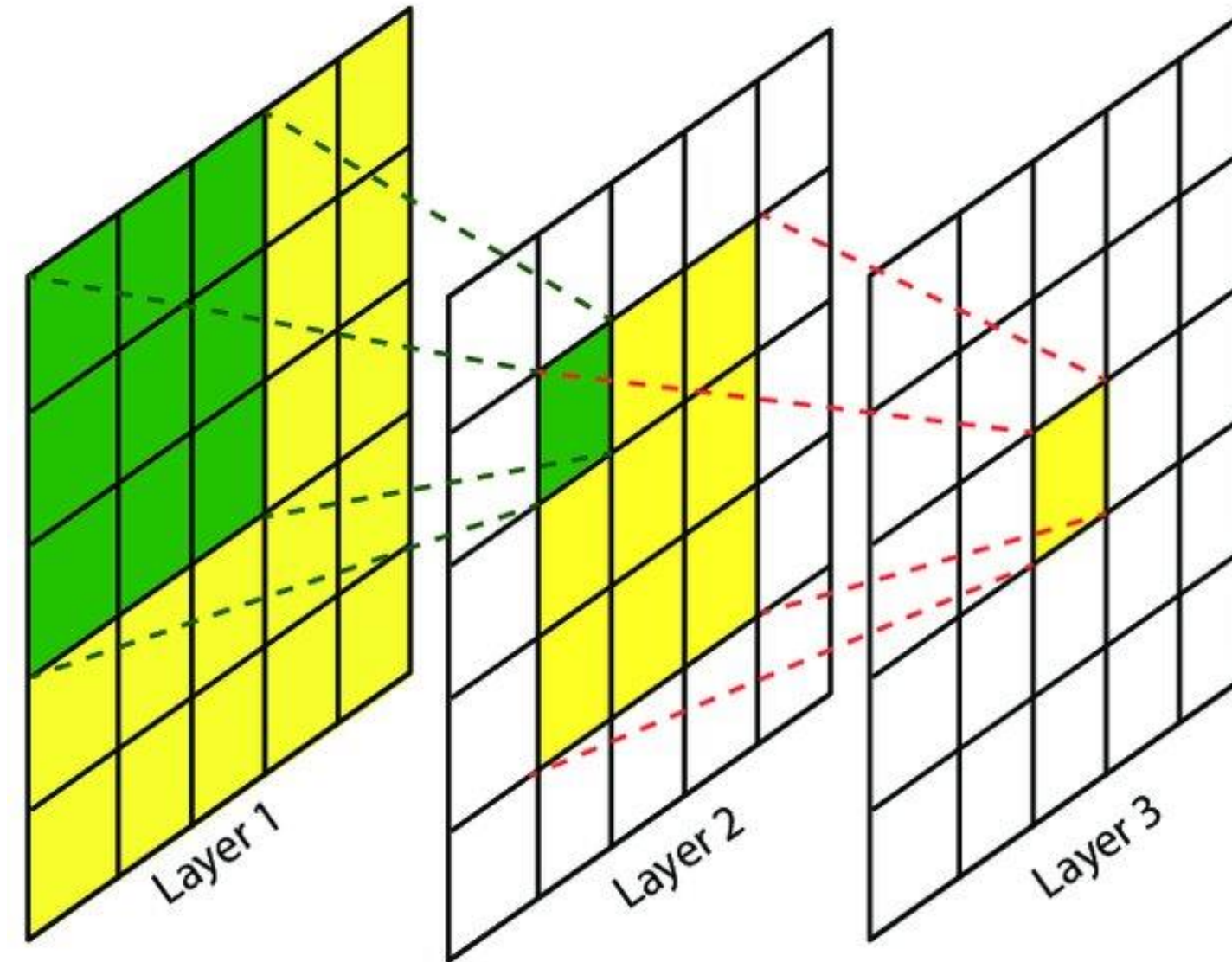
These convolutional layers are repeated





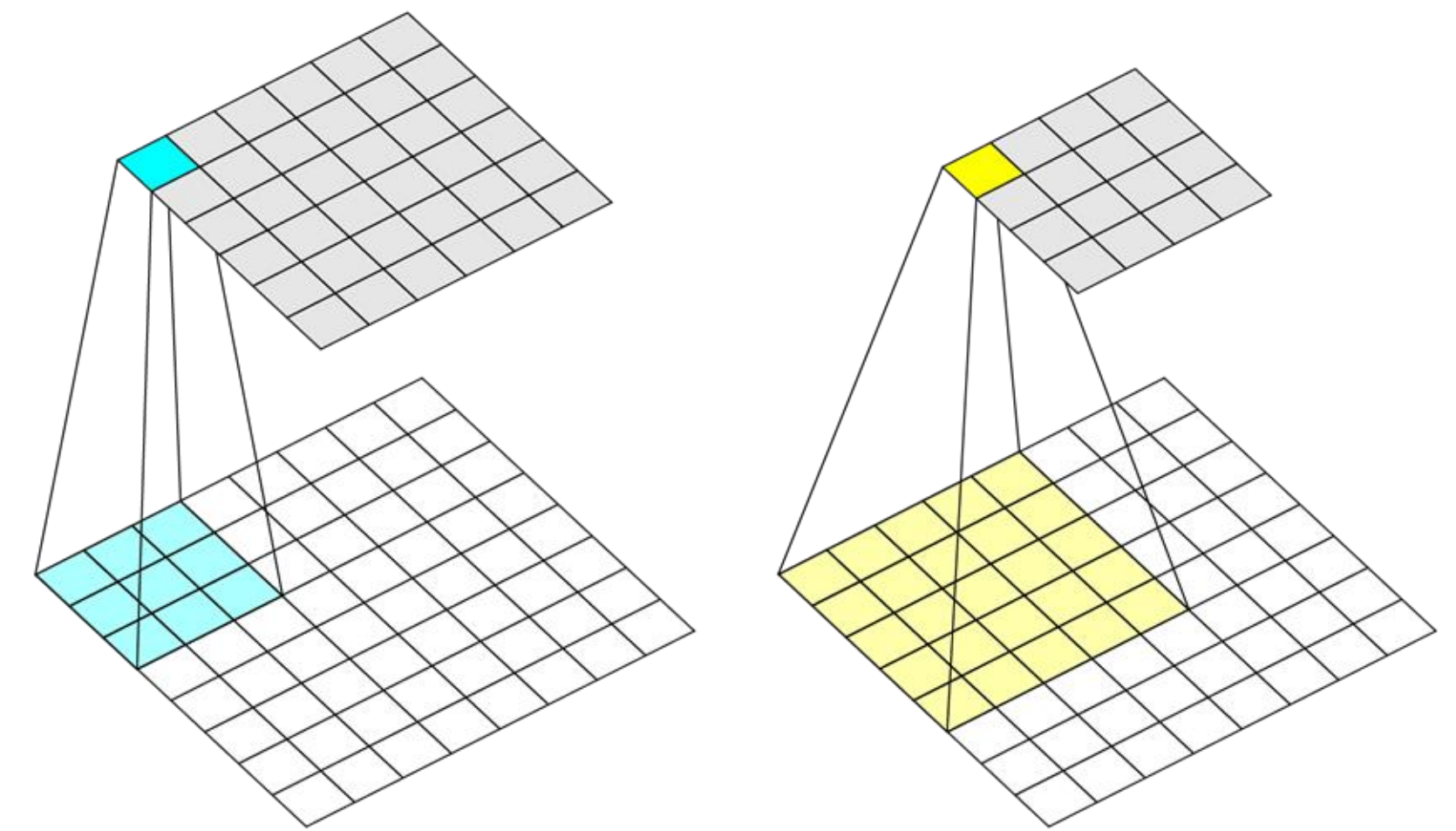
# From Local to Global

- The receptive field is defined as the region in the input space that a particular CNN feature is looking at (i.e. be affected by)
- Increases as we go deeper in the network

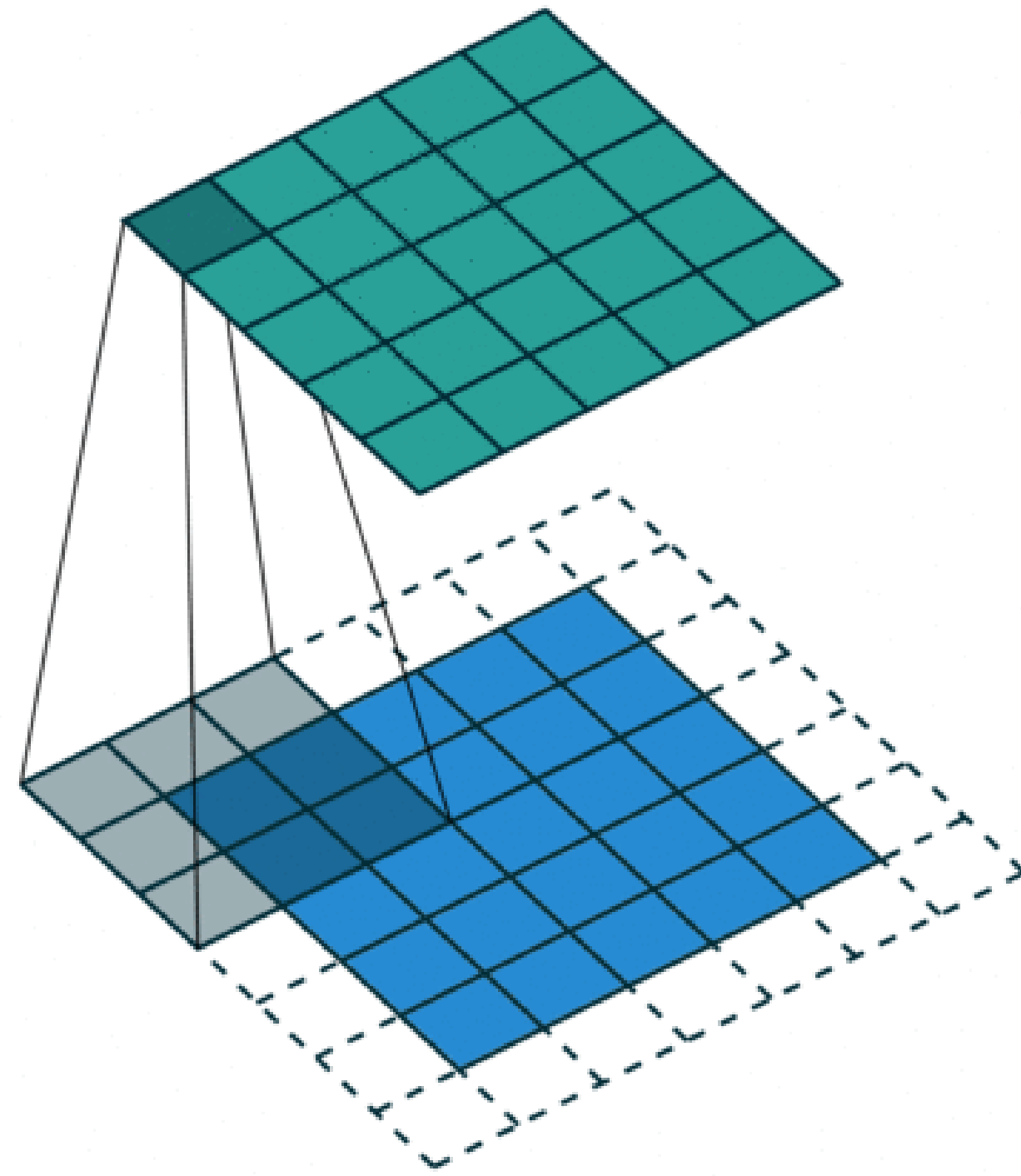


# Convolution Layer

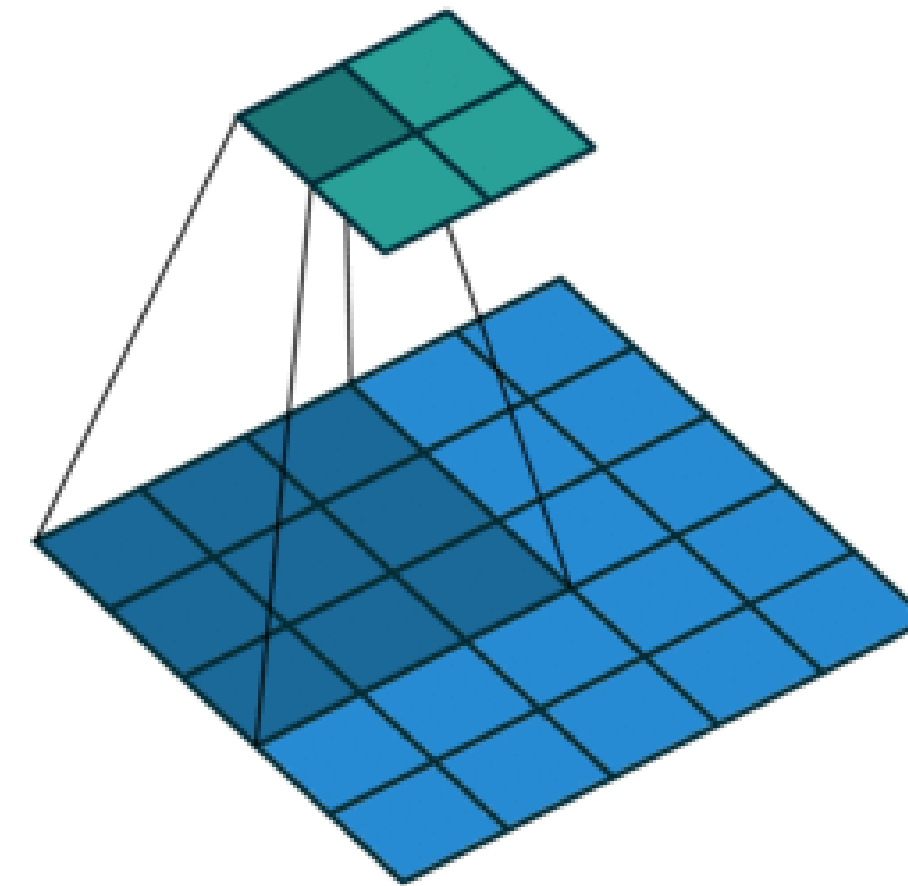
- **Input:** “an image”  
**Processing:** convolution with multiple filters  
**Output:** an image, # channels = # filters
- Kernel size
  - Typically  $1 \times 1 \leftrightarrow 11 \times 11$
  - $1 \times 1$  is just linear combination of channels in previous image (no spatial processing)



# Padding and Stride



padding - avoids shrinking  
due to the border pixels



strided convolutions - use a step  $> 1$   
( usually the same for all dimensions,  
but mandatory )



# Hyperparameters\*

- **# of filters**: integer indicating the # of filters applied to each window
- **kernel size**: tuple (width, height) indicating the size of the window
- **stride**: tuple (horizontal, vertical) indicating the horizontal and vertical shift between each window.
- **padding**: boolean to indicate if the input is padded with a border of zeros to ensure that the output has the same size as the input.

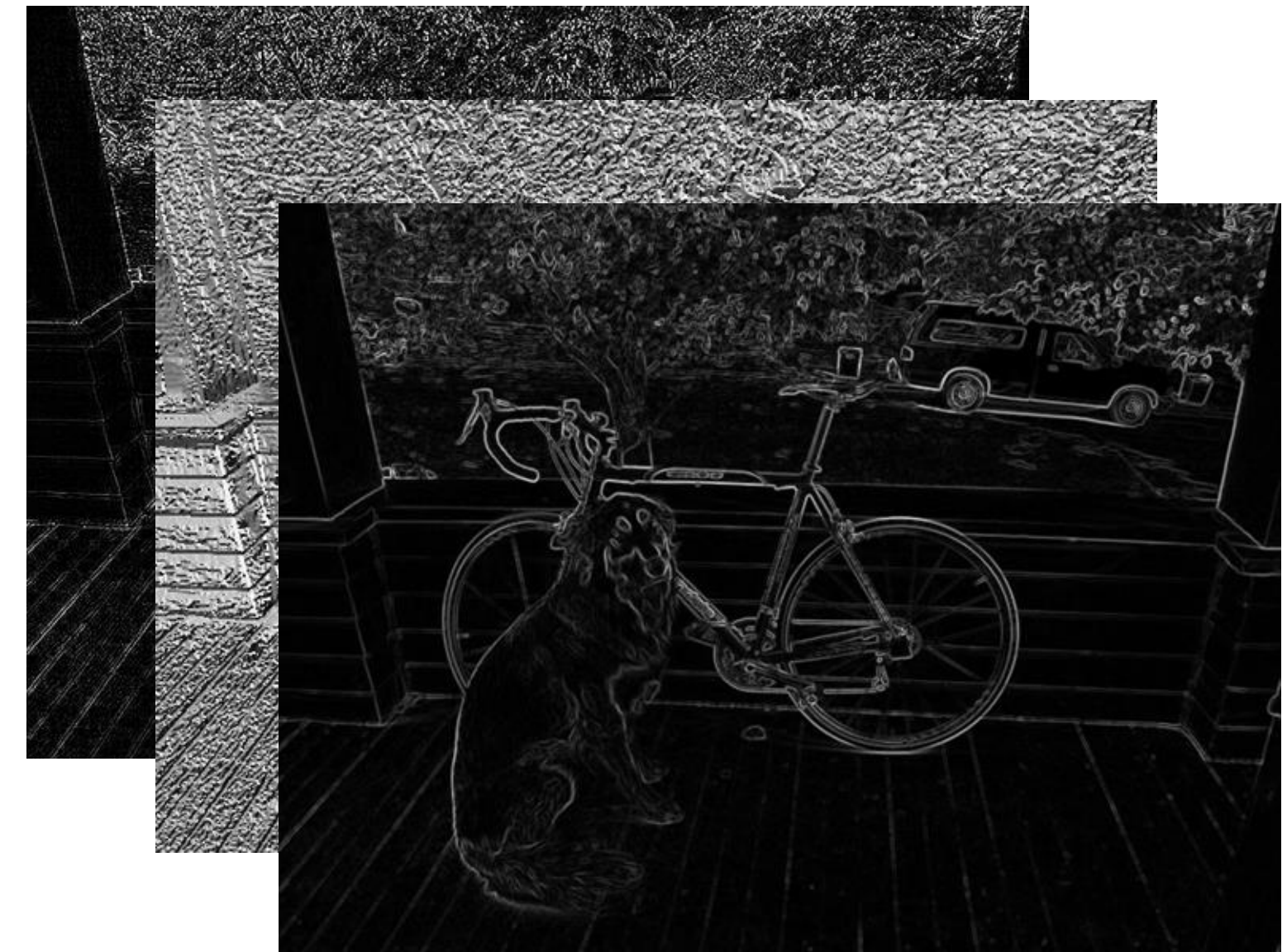
# Exercise

- Calculate the **output volume size** {width, height, channels} of a convolutional layer composed of 10 filters of size 3x3 without padding and stride of 1, when applied to a color input image of size 6x6.
- Also, **how many parameters** (learnable) does this layer have?

# Images Dimensionality

- Even a 256 x 256 images has hundreds of thousands of pixels and that's considered a small image!

- Convolution:



- Aggregate information, maybe we don't need all of the image, can subsample without throwing away useful information



# Pooling

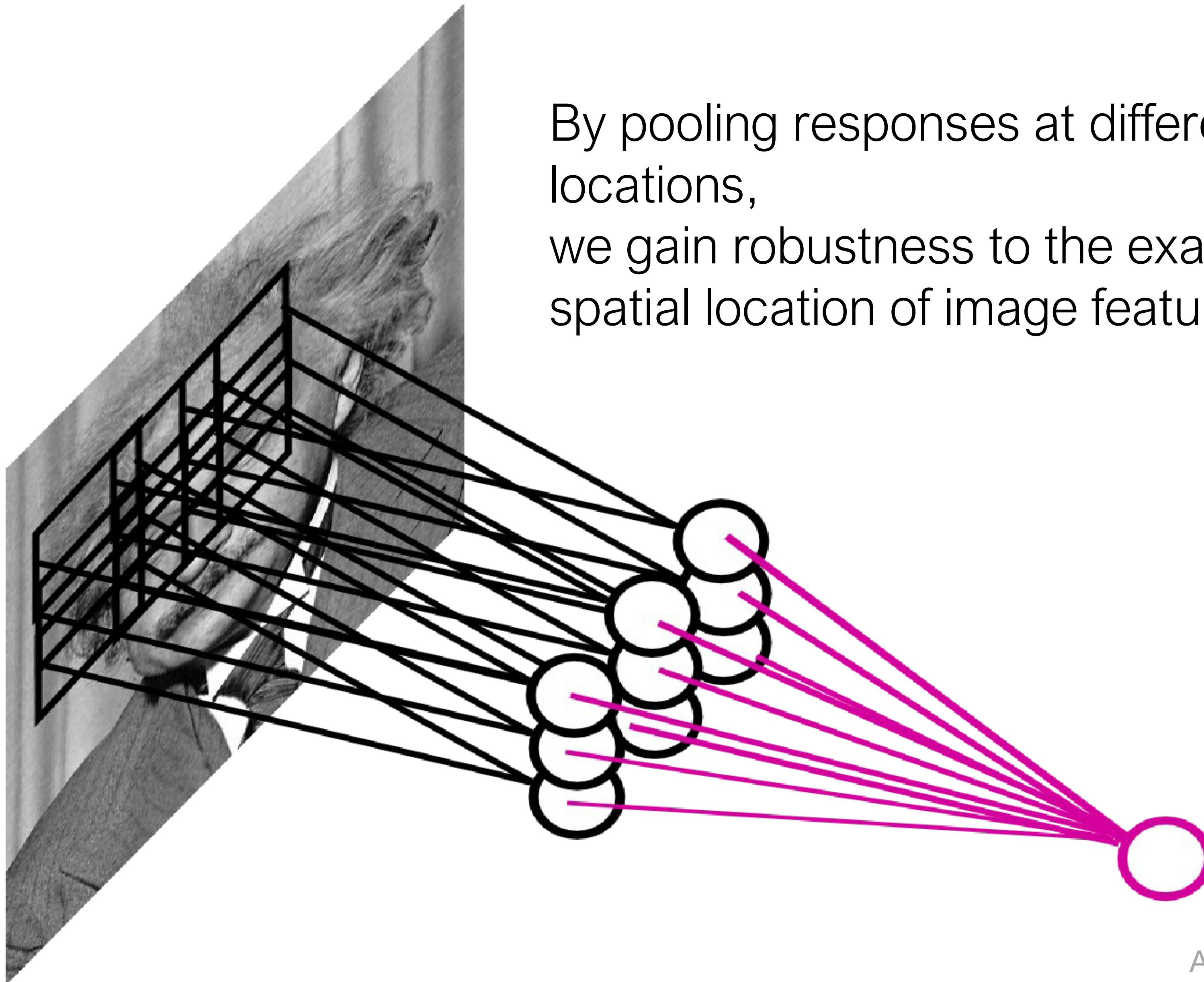
- Pooling: **commutative** mathematical operation that combines several units
  - Examples: max, sum, product, average, Euclidean norm, etc.
  - Commutative property (order does not matter)

$$\max(a, b) = \max(b, a)$$

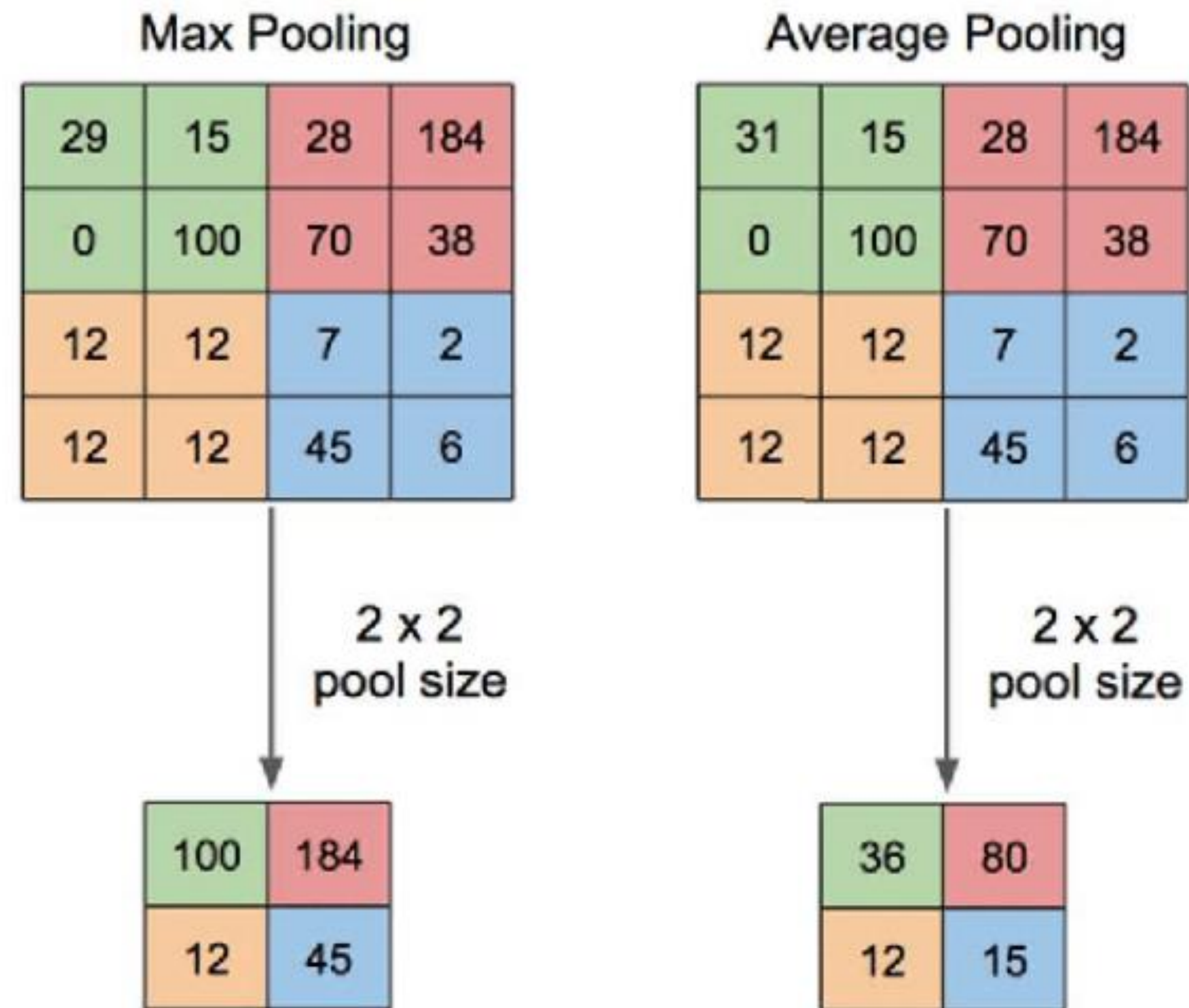
- Makes the representations smaller and more manageable
- Also, more robust to location

# Pooling Layer

By pooling responses at different locations, we gain robustness to the exact spatial location of image features.



# Pooling Layer



The operations are done per channel → same depth as input

How many parameters?  
And how many hyperparameters?



# Pooling Layer

**Input:** an image

**Processing:** pool pixel values over region

**Output:** an image, shrunk by a factor of the stride

Hyperparameters:

- What kind of pooling? Average, max, min

- Stride? Controls downsampling

- Size? Usually the same or not much bigger than stride

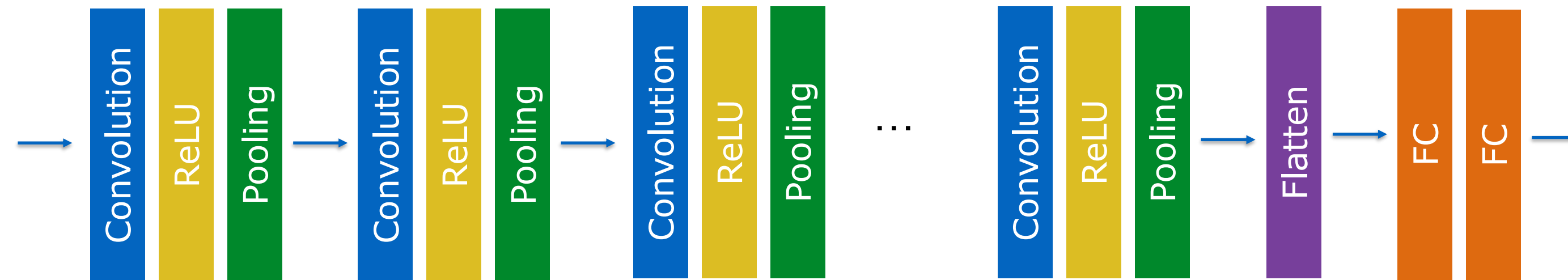
Most common: 2x2 or 3x3 maxpooling, stride of 2

# Building blocks of a CNN

- **Convolutional layers:**  
Connections are convolutions  
Used to extract features
- **Pooling layers:**  
Used to downsample feature maps, make processing more efficient  
Most common: maxpool, avgpool sometimes used at end
- **Connected layers:**  
Often used as last layer, to map image features -> prediction  
No spatial information  
Inefficient: lots of weights, no weight sharing

# Building blocks of a CNN

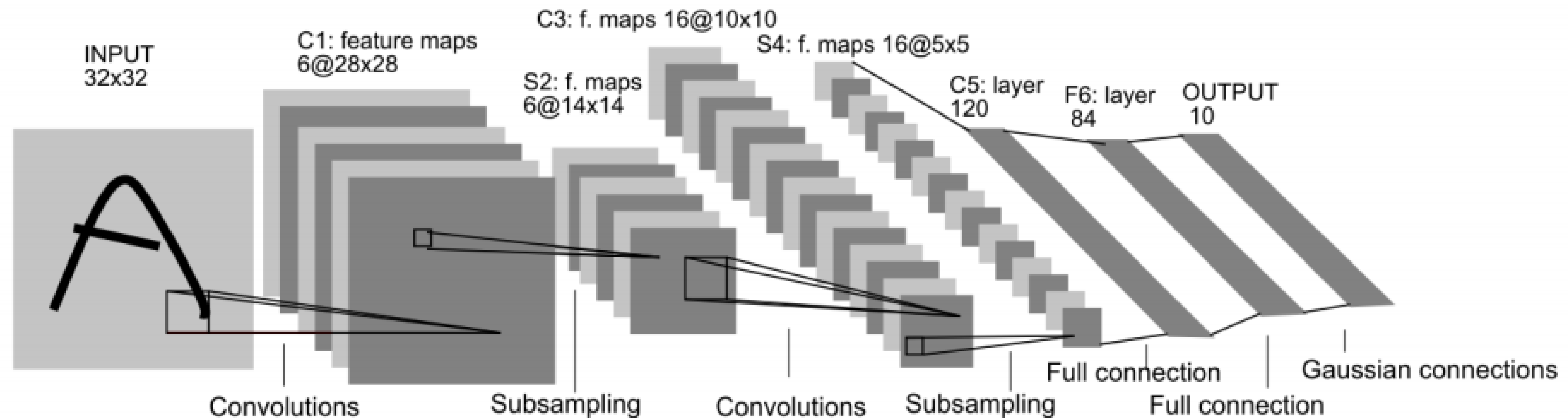
- Stacking Convolutional layers we can build a NN with several interesting properties.



- Inserting a non-linear activation function we achieve non-linear **complex transformations**.
- Inserting pooling layers we can down-sample the spatial information and **trade with more feature filters** learnt during training.
- Each new layer learns on top of previously filtered images with **less spatial information and more extracted features**.
- Inserting a final classifier (fully-connected MLP) over a “flattened” image allows to do classification or regression, not on the raw pixels but on **learnt features of the image**.



# LeNet 5, LeCun 1998



- Input: 32x32 pixel image. Largest character is 20x20 (All important info should be in the center of the receptive field of the highest level feature detectors)
- Cx: Convolutional layer
- Sx: Subsample layer
- Fx: Fully connected layer